MIPS16e Field Manual: Chapters 5-7 Expanded

Control Flow, Functions, and Ghidra Reverse Engineering | Little-Endian MIPS32 / MIPS16e

Independent Study Companion | Technical revision: November 2026

Contents

Read this first: what this companion does 3

CHAPTER 5. Control flow in mixed MIPS16e/MIPS32 4
- 5.A. What a branch actually compares . . . 4
- 5.B. Delay slots: three distinct rules, not one . . . 4
- 5.C. Branch displacements: byte address versus encoded count . . . 5
- 5.D. Comparison via T (`$t8`) and condition branches . . . 5
- 5.E. Jump, call, return: the role of `$ra` . . . 6
- 5.F. Mode changes: an address is not sufficient . . . 6
- 5.G. Control-flow graphs: reconstruct an `if` statement . . . 7
- 5.H. Classic MIPS32 comparison: the source of many mistakes . . . 8
- 5.I. Deep-dive notebook: how to trace any control transfer . . . 8
- 5.J. Chapter 5 applied lab: three paths to one result . . . 8

CHAPTER 6. Functions, stack frames, ABIs, and PIC 9
- 6.A. Three different systems inside one function call . . . 9
- 6.B. Caller versus callee: the physical register problem . . . 9
- 6.C. Callee-saved: how the contract is enforced . . . 10
- 6.D. Function arguments are values, not entire objects . . . 11
- 6.E. Stack versus registers: the mental model . . . 11
- 6.F. Frame pointer (`$fp`) versus stack pointer (`$sp`) . . . 12
- 6.G. Leaf, nonleaf, and nested calls . . . 12
- 6.H. MIPS16e SAVE and RESTORE: why they matter . . . 13
- 6.I. Position-independent code: PIC, `$t9`, and `$gp` . . . 13
- 6.J. Tail calls: when a function does not return to itself . . . 14
- 6.K. Function pointers and indirect calls . . . 14
- 6.L. Why a decompiler can guess the wrong function prototype . . . 15
- 6.M. A complete function trace with a status pointer . . . 15
- 6.N. Chapter 6 mastery problems . . . 16

CHAPTER 7. Machine-code decoding and Ghidra investigation 17
- 7.A. Four layers that must never be confused . . . 17
- 7.B. Little-endian byte reading: data word versus instruction halfwords . . . 17
- 7.C. Decode a real 16-bit MIPS16e BEQZ field by field . . . 17
- 7.D. Decode a negative branch offset: sign extension in action . . . 18
- 7.E. Decode a PC-relative load: literal pools and hidden constants . . . 19
- 7.F. EXTEND: two halfwords, one instruction . . . 19
- 7.G. Decode a classic MIPS32 instruction for comparison only . . . 20
- 7.H. Ghidra mode selection: decode before decompile . . . 20
- 7.I. Functions, symbols, and XREFs: what each one can prove . . . 21
- 7.J. Backward slicing: find the actual pointer base . . . 21

7.K. Trace `base + 0x204` without inventing a struct . . . 22
7.L. Two-step indirection: pointer stored near the PC . . . 23
7.M. Search strategies beyond one literal immediate . . . 23
7.N. Structures, arrays, and memory-mapped registers . . . 23
7.O. P-code: why it helps when assembly syntax varies . . . 24
7.P. A practical Ghidra investigation session . . . 24
7.Q. A short Python decoder for two verified MIPS16e branch forms . . . 25
7.R. Anti-patterns: eight ways to fool yourself . . . 25
7.S. Chapter 7 mastery problems . . . 26

APPLIED CASE STUDIES: Put Chapters 5-7 together **27**
Case 1. Recover a branch from two bytes . . . 27
Case 2. Why MIPS16e and classic MIPS32 have different traces . . . 27
Case 3. Reconstruct a stack frame before naming locals . . . 27
Case 4. A nested call overwrites the return link . . . 28
Case 5. Trace a PC-relative pointer into a structure-like field . . . 28
Case 6. An XREF you do not see . . . 29
Case 7. A mixed-mode call and an apparently odd return pointer . . . 29
Case 8. A 4-byte instruction is mistaken for two instructions . . . 29
Case 9. One access, four explanations . . . 30
Case 10. Write a defensible research note . . . 30

APPENDIX A. Fast reference cards **32**
A.1. Control-flow card . . . 32
A.2. Function/stack card . . . 32
A.3. Byte/decode card . . . 32
A.4. Thirty-second Ghidra checklist . . . 33

APPENDIX B. Worked answers and diagnostic explanations **34**
B.1. Chapter 5 checkpoints . . . 34
B.2. Chapter 6 checkpoints and mastery . . . 34
B.3. Chapter 7 checkpoints and mastery . . . 35
B.4. Independent variants from the ten case studies . . . 35
B.5. A final ten-question readiness exam . . . 36

SOURCES, SCOPE, AND VERIFICATION **36**

Read this first: what this companion does

Purpose. This is a deeper companion to *Lessons 5, 6 and 7* of your existing textbook. It is not a replacement for your basic memory, hexadecimal, register, or load/store lessons. It assumes you can already explain `lw`, `sw`, `andi`, sign extension, an offset, and a pointer.

Your established target. Architecture: **MIPS32**. Byte order: **little-endian**. Code under investigation: **MIPS16e**. Instruction lengths: **2 or 4 bytes**. Ghidra processor language: **`MIPS:LE:32`**. These are the working inputs to this manual, not evidence that every byte in the entire firmware image uses MIPS16e.

Unconfirmed details. The processor's exact revision, firmware calling convention, symbol availability, address maps, privilege behavior, and tool-version-dependent Ghidra user interface remain unknown. When this manual uses **O32**, it is explaining a *common ABI*, not asserting your firmware has been proved to use it.

How to read quickly. Each section starts with a **one-sentence rule**, then an **execution trace**, then a **failure mode**, then a **checkpoint**. Bold text marks the ideas to retrieve from memory. Code samples are marked **MIPS16e**, **classic MIPS32**, or **schematic**. Never treat a schematic sequence as verified machine code.

Crucial distinction: a *32-bit-wide instruction in MIPS16e mode* is **not automatically an ordinary MIPS32 instruction**. The former can be two 16-bit halfwords interpreted by the MIPS16e decoder. [M1, §§3.14-3.17]

Read in this order	Main question answered	Expected output
Chapter 5, A-F	Which instruction executes next?	Correct branch/call trace including ISA mode
Chapter 6, A-F	Which values survive a function call?	Hand-drawn stack frame and argument trace
Chapter 7, A-H	What do these raw bytes mean?	Byte-to-instruction decode and Ghidra evidence chain
Case studies and answer key	Can I apply the rules independently?	Solved examples, then your own answers

> **Evidence ladder: Observed bytes** are facts. **A verified decode** is an architectural interpretation. **A decompiler type** may be a guess. **A guessed function purpose** is a hypothesis. **An instruction existing in firmware** does *not* prove it executed during a particular event.

Notation: PC = instruction address; RA/`$ra` = link or return register; SP/`$sp` = stack pointer; GPR = general-purpose register; `sext` = sign extension; `zext` = zero extension; & = bitwise AND; << = left shift; `[ ]` = memory access in diagrams; `0x` = hexadecimal. Hex addresses used in exercises are invented unless identified as your established target example.

CHAPTER 5. Control flow in mixed MIPS16e/MIPS32

5.A. What a branch actually compares

Rule: In `beq rs, rt, destination`, the **first two operands are compared** and the **last is where execution would go**.

Classic MIPS32 teaching example:

```
# Classic MIPS32 teaching example, not MIPS16e bytes
# $t0 = 0, $zero = 0
0x2000: beq $t0, $zero, 0x2010
```

Because `$t0 == $zero`, the branch is taken. **`0x2010` is not compared with `$zero`**: it is a destination. This corrects the common confusion between *the branch condition* and *the branch target*.

In MIPS16e, compact conditional instructions include BEQZ, BNEZ, BTEQZ, and BTNEZ. An ordinary `BEQZ rx, target` tests whether a named register contains zero; BTEQZ tests the special T condition register (architectural `$t8`). [M1, §§3.6, 3.13 and instruction descriptions]

Vocabulary: taken means condition true, so control transfers to the branch target; **not taken** means execution continues at the next instruction. **Fall-through address** means the next instruction boundary if a transfer is not taken.

Checkpoint 5.A: With `$v0=0`, does `BEQZ $v0, L` branch because L is zero, or because `$v0` is zero? Answer in the back.

5.B. Delay slots: three distinct rules, not one

Rule: Never infer a delay slot from the word 'branch'. The **actual instruction and ISA mode** determine whether a following instruction executes before transfer.

Mode/instruction	Delay slot?	Reasoning consequence
Classic MIPS32 BEQ, BNE (pre-R6 conventional model)	Yes	Immediate following instruction executes on taken and untaken paths
Classic MIPS32 J, JAL, JR	Yes	Delay instruction executes before reaching target
MIPS16e B, BEQZ, BNEZ, BTEQZ, BTNEZ	**No**	If taken, following instruction does not execute merely as a delay slot
MIPS16e JAL, JALX, JALR, JR	**Yes**	Exactly one instruction delay, with restrictions
MIPS16e JALRC, JRC	**No**	Compact link/jump transfer without a delay slot

These distinctions are specified in [M1, §3.13]. **Do not apply classic delay-slot reasoning indiscriminately to MIPS16e.**

Trace A: No delay slot (MIPS16e)

```
# Addresses are schematic, but control-flow rule is real.
0x2000   BEQZ v0, 0x2008       [2 bytes]
0x2002   instruction_X         [2 bytes]
0x2004   instruction_Y         [2 bytes]
0x2006   instruction_Z         [2 bytes]
0x2008   target_instruction    [2 bytes]
```

If `$v0=0`, branch **taken**: the next instruction executed is at `0x2008`; `instruction_X` is not an automatic delay-slot operation. If `$v0=9`, branch **not taken**: next execution is `0x2002`.

Trace B: Delayed jump (MIPS16e)

```
0x3000   JAL helper          [4 bytes]
0x3004   safe_slot           [2 bytes]
0x3006   after_call          [2 bytes]
...      helper:
```

The `safe_slot` at `0x3004` executes **before the callee begins**. The MIPS16e JAL instruction records the return address **`0x3007` in `$ra`**, because its ordinary aligned continuation is `0x3006` and bit 0 tags MIPS16e mode. This is **not** classic MIPS32's PC+8 formula. [M1, JAL instruction, p. 83]

Restrictions: A jump/branch or an extended instruction is not a valid general-purpose filler for MIPS16e jump delay slots; the manual describes such configurations as **UNPREDICTABLE**. Inspect both instruction length and slot legality. [M1, §3.13]

Checkpoint 5.B: If an ordinary two-byte instruction sits immediately after a taken MIPS16e BEQZ, does it automatically execute? How would the answer differ for a permitted instruction immediately after MIPS16e JAL?

5.C. Branch displacements: byte address versus encoded count

Rule: In nonextended MIPS16e BEQZ/BNEZ, the encoded displacement counts **2-byte halfwords**, not bytes or 4-byte words.

For a **nonextended 16-bit** BEQZ instruction:

```
branch target = branch_PC + 2 + (sign_extend(offset8) << 1)
```

The +2 means advance past the current 2-byte instruction. The <<1 means multiply the signed displacement by 2. [M1, BEQZ, p. 68]

Forward example: At PC `0x2000`, encoded offset = +3:

```
base                0x2002
three halfwords     +0x0006
result              0x2008
```

Backward example: At PC `0x2100`, encoded offset = `-4`:

```
base                0x2102
negative 4 * 2      -0x0008
result              0x20FA
```

A backward branch often implements a loop. But a backward branch alone does not prove what condition causes loop termination; analyze the surrounding operations.

Do not transplant this formula to every instruction. An *extended* MIPS16e BEQZ is **4 bytes** and uses `PC + 4 + (signed 16-bit displacement << 1)`; classic MIPS32 BEQ uses `PC + 4 + (signed 16-bit displacement << 2)`. [M1, BEQZ pp. 68-69]

Form	Instruction bytes	Address base	Encoded unit
MIPS16e short BEQZ	2	PC+2	2-byte halfwords
MIPS16e extended BEQZ	4	PC+4	2-byte halfwords
Classic MIPS32 BEQ	4	PC+4	4-byte words

Checkpoint 5.C: A nonextended MIPS16e BEQZ is at `0x1200` with signed displacement `-3`. Give its target address, then say which instruction executes next if the tested value is nonzero.

5.D. Comparison via T (`$t8`) and condition branches

Rule: Some MIPS16e comparisons write the **T register**, also known as architectural `$t8` (`$24`), and subsequent BTEQZ/BTNEZ instructions test that result.

```
# MIPS16e conceptual trace; check exact forms in the disassembler
cmp   $a0, $a1
bteqz equal_path
```

For CMP, the comparison value is the bitwise XOR of operands. Thus equal bit patterns yield zero. For $a0=0x0F and $a1=0x14, T=0x1B (27 decimal), not zero. A following BTEQZ is not taken. For $a0=$a1, T is zero and BTEQZ takes its branch. [M1, MIPS16e register set and CMP/BTEQZ instructions]

Important distinction: BTEQZ does not compare its branch target with zero. Like BEQZ, its target is control-flow data; its test is the T register.

From conditional to C-like logic

```
load field
compare against expected value
branch if equal -> success block
fall through -> failure block
```

The disassembler often gives a branch target, not source-level `if` or `else` boundaries. You reconstruct the structure by **following both successors** and the blocks that eventually rejoin.

Checkpoint 5.D: If T contains 0x00000004, does BTEQZ branch? Would BTNEZ branch?

5.E. Jump, call, return: the role of $ra

Rule: A branch changes control flow; a call also records where to come back.

Operation	Main effect	Key register
Unconditional branch	Transfer without saving a return point	None required
JAL	Call in the same ISA mode and set a return link	$ra
JALX	Call and exchange ISA mode	$ra plus mode
JALR / JALRC	Call using an address held in a register	$ra
JR / JRC returning via $ra	Transfer back to caller (and recover tagged mode when appropriate)	$ra

A function call is not a separate hardware stack. If a function calls another function, the new link normally overwrites $ra. To preserve its own return destination, a nonleaf function commonly saves $ra in stack memory before making its nested call.

Example: A calls B; B calls C. If B never preserves its incoming $ra before calling C, the return address back to A may be lost. This is the essential reason that many function prologues save $ra.

The **ABI** tells us which registers/stack positions carry arguments and which registers must survive a call; the **ISA** tells us what JAL, JR, SW, and other operations actually do. Keep these layers separate.

5.F. Mode changes: an address is not sufficient

Rule: On a MIPS16e-capable processor, **instruction-set mode is part of the control-flow state**.

State	Meaning
ISA mode 0	Ordinary 32-bit MIPS decoding
ISA mode 1	MIPS16e decoding

The architecture uses **bit 0** of certain link and indirect jump address values as an **ISA-mode tag**. The hardware clears that bit for instruction fetch; it does **not** fetch a halfword from an odd byte address. JALX changes modes; the appropriate register-indirect jumps/calls can also select mode using the low bit of their target register. [M1, §§3.7-3.8]

```
Actual code entry 0x80001200, MIPS16e:
  tagged function pointer = 0x80001201
  fetch address           = 0x80001200
  ISA mode                = 1

Actual code entry 0x80002000, classic MIPS32:
  ordinary mode pointer   = 0x80002000
  fetch address           = 0x80002000
  ISA mode                = 0
```

Real implication for Ghidra: Correct bytes decoded in the **wrong ISA mode** can look like nonsense or even plausible but false assembly. Selecting `MIPS:LE:32` is **not sufficient by itself** to establish the ISA mode at every address. [G1, G2]

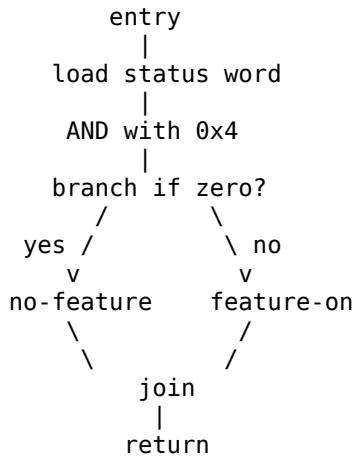
A call and return across modes

```
A: MIPS16e code at 0x80001000
      |
      | JALX  (switch mode, save MIPS16e-tagged return)
      v
B: classic MIPS32 code at 0x80002000
      |
      | return using appropriate register jump
      v
A: MIPS16e code resumed, because return pointer encodes mode
```

When exploring mixed code, record **both** the aligned address **and** its mode. If the decompiler's function boundaries cross incompatible ISA modes, inspect the boundary rather than trusting the inferred call graph.

5.G. Control-flow graphs: reconstruct an `if` statement

Rule: A *basic block* is an instruction sequence with one entry and a known exit decision; a *control-flow graph* (CFG) links the blocks through possible transitions.



A mask of `0x4` asks about **bit 2**. But **the bit's semantic meaning is not known** until corroborated by callers, data structures, runtime observation, or symbols. The branch only proves a test and two paths.

Four common CFG shapes

1. **If/else:** a conditional branch with two successors that may rejoin.

2. **Loop:** an edge goes backward to a prior block; identify which block changes the loop variable.
3. **Early return:** a conditional path exits the function, often after a check.
4. **Dispatch/switch:** a computed target or series of comparisons selects among many blocks; indirect dispatch can hide static XREFs.

Checkpoint 5.G: A BNEZ follows `andi v0,v0,0x04`. What is known about the condition, and what is *not* known about the real-world meaning of the bit?

5.H. Classic MIPS32 comparison: the source of many mistakes

Rule: The same mnemonic family may have **different encoded widths, offset units, and delay behavior** in MIPS16e and classic MIPS32.

Classic teaching example:

```
# PRE-RELEASE-6 CLASSIC MIPS32, NOT MIPS16e
0x1000: beq   $t0, $zero, 0x1010
0x1004: addiu $t1, $t1, 1   # delay slot
0x1008: addiu $t2, $t2, 1   # fall-through
0x1010: addu  $v0, $t1, $t2
```

If `$t0=0`, the delay-slot increment at `0x1004` **still runs**, whereas `0x1008` is skipped. This pattern should **not** be copied unchanged when you see a MIPS16e BEQZ.

Trap: 'PC + 4 always'. That is not a general MIPS16e rule. In compressed code, an ordinary next instruction may be at PC+2; an extended instruction may be 4 bytes; a link may encode a mode bit. Calculate from the **verified form**, not from habit.

5.I. Deep-dive notebook: how to trace any control transfer

Follow this six-line checklist for every suspicious branch or jump:

1. **Mode:** MIPS16e or classic MIPS32 at this address?
2. **Instruction length:** 2 or 4 bytes? Is there an EXTEND pair?
3. **Predicate:** Which register or flag is being tested, and what is its value?
4. **Fall-through:** Which address executes if the branch is not taken?
5. **Destination:** Is target absolute-region, PC-relative, register-indirect, or mode-tagged?
6. **Delay-slot rule:** Does the next instruction run before transfer? Is it legal in that slot?

   Minimum competent trace: Show *address + length + mode + register state + chosen successor*. If any of those are unknown, annotate them as unknown instead of inventing a path.

5.J. Chapter 5 applied lab: three paths to one result

All addresses below are schematic verified-length placements; the actual opcodes are supplied in Chapter 7 where hand-decoding is intended.

```
MIPS16e; initial $v0 = 0x0000000C, $s0 = 0x4000FA00
0x5000 [2] v0 <- v0 AND 0x04    # pseudocode, not a literal MIPS16e opcode
0x5002 [2] beqz  v0, 0x500A
0x5004 [2] addiu s0, 4
0x5006 [2] b     0x500C
0x5008 [2] <unreached padding>
0x500A [2] addiu s0, 8
0x500C [2] ...
```

Trace: Masking `0x0C` with `0x04` gives `0x04`. Therefore BEQZ is **not** taken; the instruction at `0x5004` runs and adds 4 to `$s0`; unconditional B at `0x5006` has **no delay slot**, so the next executed instruction is at `0x500C`, *not* `0x5008`. Final `$s0=0x4000FA04`. This is a **control-flow pseudocode trace**, not a claimed literal MIPS16e byte sequence. In particular, a one-instruction AND-immediate form must not be assumed for every base MIPS16e CPU; the mask can be computed by other instructions. Its purpose is the control decision and the next instruction, not verifying every encoding.

Check yourself: If initial `$v0=0x08`, what final `$s0` value results? Work the branches on paper before checking Appendix B.

CHAPTER 6. Functions, stack frames, ABIs, and PIC

6.A. Three different systems inside one function call

Rule: A function call involves **ISA instructions**, **calling-convention promises**, and **compiler choices**. They are related but not interchangeable.

- **ISA:** What an instruction such as `JALR`, `SW`, or `ADDIU` does to registers, memory, and control flow.
- **ABI:** Which registers a function is allowed to overwrite, where arguments are expected, and what the callee promises to preserve.
- **Compiler:** Whether a particular variable lives in `$s0`, whether a frame pointer is needed, and how large a stack frame will be.

Example: The statement "`$s0` is callee-saved" is an **O32 ABI rule**. The CPU has no special mechanism that automatically saves `$s0`; the function must implement the promise using instructions and memory if it changes the register.

A note on the target: **O32 is a useful likely-looking pattern, not yet a confirmed MediaTek firmware ABI.** Static code may follow a custom freestanding convention, contain handwritten assembly, or use helper stubs. Our job is to infer the actual contract from observed callers and callees before relying on it. [A1]

6.B. Caller versus callee: the physical register problem

Rule: Calling B does **not** give B a new private set of CPU registers. Both A and B operate on the same physical register file, so **something must preserve values needed after a call**.

Consider:

```c
int B(int n) { return n + 1; }
int A(void) {
    int important = 50;
    int result = B(7);
    return important + result;
}
```

Suppose A has its `important` variable in `$t0` (an O32 caller-saved register). B is **allowed** to use `$t0` for unrelated internal work. A therefore must either save 50, arrange to recompute it, or keep the value somewhere that survives the call.

A complete caller-save example

```
# CLASSIC MIPS32 / illustrative O32 stack sequence
# A wants t0 after B returns.
A:
    addiu $sp, $sp, -32   # Reserve 16-byte arg area and saved-value slots
    sw    $ra, 28($sp)    # Save A's incoming return link
    addiu $t0, $zero, 50
    sw    $t0, 24($sp)    # Caller protects its live t0
    jal   B
    nop                  # Classic delayed call slot
    lw    $t0, 24($sp)    # Original 50 recovered by A
    addiu $v0, $t0, 1
    lw    $ra, 28($sp)
    addiu $sp, $sp, 32
    jr    $ra
    nop
```

`sw $t0, 24($sp)` means **copy register `$t0` into the stack memory at address `$sp`+24**. It does **not** mean loading memory into a register. `lw $t0, 24($sp)` reverses the direction. **B does not have to restore `$t0`.**

This O32-style teaching example reserves a **16-byte outgoing-argument area** at the low end of a 32-byte stack frame, leaving higher offsets for the saved values. Exact stack conventions differ across ABIs and toolchains. The stack pointer must be valid and adequate stack space reserved; arbitrarily writing at `0($sp)` without allocation is unsafe.

What if A never uses the value again?

Then **A need not save `$t0` for that reason**. 'Caller-saved' means the *caller bears the responsibility if it cares*. It does not mean each caller must always push every temporary register onto the stack. Optimized code often avoids such unnecessary stores.

6.C. Callee-saved: how the contract is enforced

Rule: Under O32, a called function that modifies `$s0` must restore the **value it received** before returning; the register is not exclusively for storing other special registers.

```
# CLASSIC MIPS32 / simple stack demonstration
B:
    addiu $sp, $sp, -8
    sw    $s0, 4($sp)     # Preserve caller's s0
    addu  $s0, $a0, $a1  # B reuses s0 for its own computation
    addu  $v0, $s0, $zero
    lw    $s0, 4($sp)     # Restore exact incoming s0
    addiu $sp, $sp, 8
    jr    $ra
    nop
```

Why doesn't A save `$s0` instead? It *could* save extra state privately, but the ABI's point is to establish a **predictable contract**. A relies on `$s0` surviving; B agrees to preserve it when using that register. This permits separate compilers and libraries to call one another without coordinating each individual function's internal register choices.

Register set, typical O32	Convention	Practical interpretation
`$t0-$t9`	Caller-saved	Callee may overwrite; caller saves any live value it needs
`$a0-$a3`	Caller-saved argument registers	Callee may use/modify while handling arguments
`$v0-$v1`	Result registers	Return data normally comes back here
`$s0-$s7`	Callee-saved	Restore entry value if used
`$sp`	Preserved	Must be restored to expected entry state on return
`$ra`	Holds link, can be overwritten by nested calls	Nonleaf code normally preserves its incoming return link
`$gp`, `$t9`	ABI/PIC-specific roles	Inspect caller/callee setup, avoid universal assumptions

Register name alone is not proof of function role. A disassembler might show `$s0` but it could be a base pointer, arithmetic accumulator, or current array element. The *preservation rule* concerns entry/exit values, not semantic variable names.

6.D. Function arguments are values, not entire objects

Rule: Under conventional O32 integer/pointer calling rules, the first four argument **words** ordinarily travel in `$a0` through `$a3`.

```
int f(int x, int y, int z, int q);
int r = f(11, 22, 33, 44);
```

```
Before the call (typical O32 integer case):
    a0 = 11      first argument word
    a1 = 22      second argument word
    a2 = 33      third argument word
    a3 = 44      fourth argument word
```

Argument word means a **32-bit slot**, not a sentence and not a command-line 'word'. A pointer is 32 bits on this MIPS32 target, so one pointer typically fits in one argument word. A 64-bit integer may consume **two words** and may have alignment requirements. Floating-point rules differ with ABI variants and hard-float/soft-float conventions. [A1]

`execve(path, argv, envp)` has three C arguments: the path pointer, the argument-vector pointer, and the environment-vector pointer. The array `argv` is **not copied wholesale into `$a1`**. `$a1` holds a **pointer** to the array elsewhere in memory.

What about a fifth argument?

A common O32 pattern places additional argument words in stack slots agreed by caller and callee. There is also an argument home area in conventional O32 stack conventions. **Exact frame offsets cannot be inferred from 'fifth argument' alone** without knowing the call-site SP, ABI variant, and stack layout.

How a struct pointer is passed

```
struct Status { int mode; int flags; };
int read_flags(struct Status *p) { return p->flags; }
```

```
At call: a0 = address of struct
In callee: read 32 bits at a0 + 4
```

A plausible implementation might ultimately involve `lw ... , 4(base)`. **That load proves an address calculation and a width, not the source type.** An `int[]` or unrelated byte buffer can generate the same memory access. This is why we trace the base pointer's provenance.

6.E. Stack versus registers: the mental model

Rule: `$sp` holds a **memory address** marking the current stack area; `sw` and `lw` move values between registers and stack memory.

Typical MIPS stacks grow toward **lower addresses**.

```
higher addresses
    0x80001000    (SP at function entry)
         |
         | allocate 16 bytes
         v
    0x80000FF0    (SP after subtracting 16)

Layout for example frame:
    0x80000FFC    saved RA          SP + 12
    0x80000FF8    saved S0          SP + 8
    0x80000FF4    local 4-byte item SP + 4
    0x80000FF0    local 4-byte item SP + 0
lower addresses
```

Important: Offsets inside a stack frame are relative to the **current value of `$sp` at that instruction**. After an allocation or deallocation, `8($sp)` refers to a different absolute address if `$sp` changed. Many reverse

engineering errors result from ignoring that SP changes during a function.

Allocate then access

```
# Classic MIPS32 teaching code
addiu $sp, $sp, -16   # SP decreases by 16
sw    $s0, 8($sp)     # Store old S0 at the new SP + 8
lw    $s0, 8($sp)     # Recover old S0
addiu $sp, $sp, 16    # SP increases by 16
```

`addiu` can use negative constants. The 'u' means no signed-overflow trap; it does **not** forbid signed immediates. If you see `addiu $sp,$sp,-32`, that is a stack allocation pattern; it isn't sign-extension instruction syntax in the abstract.

Checkpoint 6.E: Initial `$sp=0x80001000`; after `addiu $sp,$sp,-0x20`, what is the new SP? What absolute address does `sw $ra,0x1C($sp)` write?

6.F. Frame pointer (`$fp`) versus stack pointer (`$sp`)

Rule: `$sp` identifies the current stack location; `$fp` (often `$s8`/register 30 under O32 naming) can provide a **stable reference point** for a function's frame.

```
At function entry: SP = 0x80001000
Allocate 0x30:    SP = 0x80000FD0
Choose frame base: FP = 0x80000FD0
```

The compiler might choose to access a local variable as `0x10($fp)`. If it later temporarily adjusts `$sp`, that frame-pointer-relative address can remain stable. But **the compiler may omit `$fp` entirely** in optimized code. No use of `$fp` is required merely because a function has local variables.

Not a scope mechanism: `$gp` (global pointer) and `$fp` (frame pointer) do not implement C 'global scope' and 'local scope' boundaries in hardware. They are address-base conventions. Any register holding an address can act as a pointer when an instruction uses it that way.

6.G. Leaf, nonleaf, and nested calls

Rule: A **leaf** function makes no further calls; a **nonleaf** function may overwrite its incoming `$ra` by calling another function.

Leaf sketch

```
caller -> leaf_function -> return
                  |
        no nested calls
```

A leaf that doesn't modify saved registers and needs no local stack storage might not allocate a frame at all. **Do not assume every function begins by subtracting from SP.**

Nonleaf sketch

```
A calls B: RA contains A's continuation
B calls C: RA gets B's continuation, replacing prior value
B must retain A's return destination elsewhere if it must return to A
```

Common code:

```
# CLASSIC MIPS32 / schematic nonleaf prologue
addiu $sp, $sp, -16
sw    $ra, 12($sp)
# ... call a helper (overwrites RA) ...
lw    $ra, 12($sp)
addiu $sp, $sp, 16
```

```
jr    $ra
nop
```

The MIPS16e **SAVE/RESTORE** instruction family can combine stack adjustment with saving and restoring registers. Thus a MIPS16e prologue may be shorter and look very different from the classic `addiu/sw` sequence. [M1, SAVE and RESTORE instruction descriptions]

Checkpoint 6.G: Why might a function preserve `$ra` even though `$ra` is 'the return-address register'? What happens if the function calls another function and discards the old return link?

6.H. MIPS16e SAVE and RESTORE: why they matter

Rule: Do not look only for separate `sw/lw` instructions when investigating MIPS16e stack frames. **SAVE/RESTORE can perform several saves/restores as one compressed instruction**.

The MIPS16e ASE defines normal and extended forms. A SAVE can establish a stack frame and store selected registers such as `$ra`, `$s0`, `$s1`, and, in extended forms, further selected registers. RESTORE reverses the operation according to its encoded selections. Exact offsets and frame size depend on the decoded fields; **do not assign an arbitrary frame size from its mnemonic alone**. [M1, SAVE pp. 123-125 and RESTORE descriptions]

```
Schematic MIPS16e function:
    SAVE {ra, s0, frame-size}      # One compact stack-setup operation
    ... work in s0 ...
    ... perhaps call another function ...
    RESTORE {ra, s0, frame-size}   # Recover state and release frame
    JRC ra                         # Non-delayed return
```

Schematic only: The braces above explain the operands and purpose; actual textual syntax is assembler-specific and Ghidra may display the operand list differently. Verify frame size against the disassembled bytes and register-state changes.

Why a debugger author cares later: Some instructions have **multi-register effects**. A naive tracer that assumes each instruction changes only one register would miss the state change. For now, simply recognize the high-level pattern.

A stack-frame reconstruction worksheet

When you see a candidate function entry, write:

```
Entry:       0x________      ISA mode: ________
Entry SP:    symbolic SP0
Frame size:  ________ (evidence: _________________)
Saved RA:    where? ________
Saved S0/S1: where? ________
Arguments:   A0/A1/A2/A3 meanings: __________
Return path: JR / JRC / other: _________________
Nested call? yes/no/unknown
```

A **symbolic SP0** is useful when you don't know the actual runtime stack address. You can still reason that an access is at `SP0 - 12` by tracking relative adjustments.

6.I. Position-independent code: PIC, `$t9`, and `$gp`

Rule: In common **MIPS O32 PIC calling sequences**, `$t9` may contain the target function's address and `$gp` may be established for access to global data. This is an **ABI/code-generation pattern**, not a universal ISA guarantee. [A1]

Why use PIC?

Position-independent code is designed to run correctly at multiple load addresses without requiring every instruction containing an address to be rewritten. Think of a shared library or a relocatable firmware com-

ponent. PIC often accesses global data indirectly through tables and base registers.

The difference between `$t9` and `$gp`

- **`$t9`:** In a common PIC convention, the call target's **entry address** is placed here before an indirect call.
- **`$gp`:** A base register used to reach certain **global/static data or entries in a Global Offset Table**.
- **Neither:** Is a special CPU mechanism that knows about C lexical scope. They are regular registers used according to software conventions.

```
# CLASSIC MIPS32 PIC sketch (not a guaranteed MIPS16e pattern)
# t9 has been loaded with the callee's entry address
jalr $t9
nop
# callee may set up gp using its entry address / ABI-specific sequence
```

Why `$t9` and not any arbitrary `$t0`? Separate compilation works because the caller and callee agree to the convention. That agreement can allow the callee to compute the appropriate $gp or use call stubs. The exact sequence changes with PIC style, linker, ABI, and MIPS16e interworking support.

GOT access versus ordinary field access

```
GP-relative load   => might fetch a pointer from a GOT/data area
base + 0x204       => might access a struct field, array entry, register map,
                      or computed memory region
```

A **GOT pointer** is not the same as the data it ultimately points at. A two-stage load may first fetch an address from global storage and then dereference that address. Track each dereference individually.

Checkpoint 6.I: Is $gp a hardware-enforced pointer to 'global scope'? Does seeing $t9 in a PIC call guarantee your unknown firmware uses O32?

6.J. Tail calls: when a function does not return to itself

Rule: A function can *finish by jumping directly to another function*, called a **tail call**; then you may see a jump instead of the expected call-and-return pattern.

```
int A(int x) { return B(x); }
```

An optimizing compiler might arrange for A to clean up its own frame, preserve the incoming return destination, and jump to B so B returns **directly to A's caller**. This can remove an extra return operation.

Reverse-engineering consequence: A branch or jump to a function-like address is not automatically a malformed call; inspect whether a prior frame is released and whether the incoming return address remains valid.

6.K. Function pointers and indirect calls

Rule: If the target is read from a register or memory, **static XREF discovery can miss some valid call relationships**.

```
int (*callback)(int) = table[index];
return callback(value);
```

Possible low-level pattern (schematic):

```
read table entry -> address in register
place integer argument in a0
perform indirect call
use v0 as return value
```

On a mixed MIPS16e/MIPS32 target, the function-pointer value may also carry an ISA-mode tag. Distinguish **the stored tagged pointer** from **the aligned instruction-fetch address** before attempting to identify a function in Ghidra. [M1, §3.7]

No XREF is not proof of no caller. A dynamic call may depend on a table populated at runtime.

6.L. Why a decompiler can guess the wrong function prototype

Rule: The decompiler's prototype is a **model of register/data flow**, not necessarily the original C signature.

Common reasons for mismatch:

1. The wrong ABI/processor language or calling convention was selected.
2. A function is entered by an unusual firmware dispatch mechanism.
3. The actual code passes pointers rather than by-value objects.
4. Arguments are optimized away, packed, or reused.
5. The code uses tail calls or specialized MIPS16e call stubs.
6. The function boundary or ISA mode was identified incorrectly.

If Ghidra shows `undefined4 f(undefined4 param_1)`, first trace which register holds `param_1`, what memory dereferences occur, and whether the function's callers prepare more arguments. Don't rename it `Status *` without corroborating evidence.

Confidence hierarchy for types

Evidence	Confidence contribution
One `lw` at offset +4	Confirms a 4-byte access only
Multiple consistent accesses at fixed offsets	Supports structured layout
Callers consistently pass one particular allocation	Supports identity of object
Readable debug symbols or matching firmware source	May strongly support semantic type
Runtime data or independent cross-processor corroboration	Supports actual role and behavior

6.M. A complete function trace with a status pointer

Schematic case, not verified target firmware. Consider a function that reads an input status word and returns whether bit 2 is set.

```
int has_flag4(const unsigned *status) {
    return ((*status & 4u) != 0u);
}
```

High-level register trace for an O32-like call:

```
Caller:
  a0 = 0x4000FC04 (address)
  call has_flag4

Callee:
  read 4 bytes from [a0] -> v0 = 0x0000000C
  v0 &= 0x00000004      -> v0 = 0x00000004
  convert result to boolean as needed -> v0 = 1
  return

Caller:
  v0 = 1 (result)
```

The pointer **`0x4000FC04` is not the status value itself**. It is the address from which the word will be read. The function can receive that pointer in an argument register without storing the entire data structure in the register.

What this does NOT prove: That this address has the same meaning in every processor's address space, that its bit 2 really means the named status, or that this function executes at a specific time. Those are independent investigation questions.

6.N. Chapter 6 mastery problems

Without reading the answers, solve the following:

1. **Preservation:** A holds `$s0=0x1234`. B wants to use `$s0`. Whose job is restoration? What value must it have after return?
2. **Liveness:** A holds `$t0=7` but never reads `$t0` after B. Does A need to preserve it for this call?
3. **Nested calls:** A calls B; B calls C. Why might B save `$ra`?
4. **Stack arithmetic:** `$sp=0x9000`, then `addiu $sp,$sp,-0x30`; B stores at `0x2C($sp)`. Compute exact absolute address.
5. **Pointer semantics:** Caller passes `a0=0x1000`, memory at `0x1000` contains `0xDEADBEEF`. Does `a0` contain the data or its location?
6. **PIC:** Does `jalr $t9` by itself prove the binary is position-independent? What additional evidence would help?
7. **Missing prologue:** A function has no initial stack subtraction. Is it necessarily not a function?
8. **MIPS16e:** Why can searching only for `sw ra,...(sp)` miss some saved-return-address sequences?

Answers and reasoning are in Appendix B. Only advance when you can explain **why** for each answer.

CHAPTER 7. Machine-code decoding and Ghidra investigation

7.A. Four layers that must never be confused

Rule: The same investigation moves through **bytes**, **decoded instructions**, **runtime values**, and **hypothesized program meaning**. They are four different things.

```
LAYER 1: RAW BYTES IN FIRMWARE
    03 22        [example MIPS16e instruction bytes]
        |
        v
LAYER 2: INSTRUCTION DECODED IN THE RIGHT ISA MODE
    BEQZ v0, +3 halfwords    [for this example]
        |
        v
LAYER 3: RUNTIME BEHAVIOR
    If v0 == 0, continue at target; otherwise fall through
        |
        v
LAYER 4: HUMAN INTERPRETATION
    Perhaps an error check / state check / return condition
    [needs supporting evidence]
```

A mistake at Layer 1 (endianness), 2 (mode/instruction boundary), or 3 (register state) invalidates later interpretation. **The decompiler cannot repair a wrong ISA mode by inventing correct high-level meaning.**

7.B. Little-endian byte reading: data word versus instruction halfwords

Rule: Your target is little-endian. A **16-bit halfword** with value `0x2203` appears in memory as bytes **`03 22`**.

```
Address        Byte
0x2000         03
0x2001         22

Halfword starting at 0x2000 = 0x2203
```

A conventional **32-bit** MIPS32 instruction `0x8E020204` appears as:

```
0x4000         04
0x4001         02
0x4002         02
0x4003         8E
```

But a **4-byte MIPS16e instruction** is organized as **two sequential 16-bit halfwords**. Its prefix/first halfword is fetched first, followed by the next halfword; do **not** rearrange them as if the first instruction field were necessarily the most significant 16 bits of a conventional MIPS32 instruction. [M1, §3.16, p. 49]

```
MIPS16e, illustrative halfword sequence (not verified as a legal instruction):

Address  Bytes   Decoded halfword
0x3000   23 F1  0xF123     first halfword
0x3002   03 22  0x2203     second halfword

These are TWO halfwords in execution order. A decoder must decide
whether they are separate instructions or one extended form.
```

An `EXTEND` prefix changes interpretation and length. If a prefix causes the following halfword to be part of the same instruction, a decoder that advances only 2 bytes after the prefix will start its next instruction **in the middle of the extended instruction**.

7.C. Decode a real 16-bit MIPS16e `BEQZ` field by field

Rule: For a nonextended MIPS16e BEQZ, the 16-bit layout is **opcode (5 bits), compact register (3 bits), signed displacement (8 bits)**. [M1, BEQZ p. 68]

Example encoded halfword:

```
0x2203 = 00100 010 00000011
         ----- --- --------
           OP   rx   offset
```

Field	Bits	Interpretation
00100	5	MIPS16e BEQZ opcode
010	3	Compact register index 2 = architectural $v0
00000011	8	Signed displacement +3 halfwords

Little-endian bytes: **03 22**. If this instruction is located at **0x2000**, target = 0x2000 + 2 + 3*2 = 0x2008.

```
0x2000: bytes 03 22  -> MIPS16e BEQZ v0, target 0x2008

v0 == 0  -> taken      -> 0x2008
v0 != 0  -> not taken  -> 0x2002
```

This is **genuine arithmetic from a documented nonextended instruction format**, not a hypothetical opcode. The branch target may be displayed by an assembler as a label instead of the numeric relative displacement.

Why compact register index 2 means $v0: The MIPS16e mapping is not the regular 5-bit MIPS32 register index. It maps 000/001/010/011/100/101/110/111 to $s0/$s1/$v0/$v1/$a0/$a1/$a2/$a3. [M1, §3.6]

Checkpoint 7.C: Decode 0x2102 as a nonextended MIPS16e BEQZ: identify opcode, compact register, and displacement. If located at 0x6000, find the taken target.

7.D. Decode a negative branch offset: sign extension in action

Rule: A signed displacement whose highest immediate bit is **1** represents a negative count. This is how short backwards loops are encoded.

Example:

```
BNEZ v0, -4 (in halfword units)

OP:        00101  [MIPS16e BNEZ]
rx:          010  [v0]
offset:  11111100  [8-bit -4]

combined: 00101 010 11111100 = 0x2AFC
little-endian bytes: FC 2A
```

If it appears at 0x2100:

```
target = 0x2100 + 2 + (-4 * 2)
       = 0x2102 - 8
       = 0x20FA
```

BNEZ takes that branch when $v0 is **nonzero**. When $v0=0, it falls through to 0x2102.

Important: The offset is *not* a hexadecimal address written into the instruction. It is a signed relative movement in **halfword units**.

Another worked arithmetic case

At PC 0x1200, offset byte 0xFD:

```
0xFD unsigned = 253
0xFD signed 8-bit = -3
sign extension preserves -3
multiply by 2 = -6 bytes
target = 0x1202 - 6 = 0x11FC
```

This is the same two's-complement principle you learned in Chapter 1, applied to a branch field.

7.E. Decode a PC-relative load: literal pools and hidden constants

Rule: MIPS16e provides PC-relative loads for nearby constants/pointers; **the register containing the address may be the PC itself**, not a `$s0` or `$gp` variable.

The nonextended MIPS16e form `LW rx, offset(pc)` uses a 5-bit opcode, compact register, and 8-bit field. The nonextended offset field is **unsigned and scaled by four**. The relevant base PC is **aligned down to a 4-byte boundary**. [M1, LW PC-relative pp. 103-104; §3.10]

Example halfword:

```
0xB204 = 10110 010 00000100
          LWPC  v0  imm8 = 4
bytes in little endian: 04 B2
```

If this instruction is at **`0x2002`** and is not in a jump delay slot:

```
BasePC before alignment = 0x2002
Aligned BasePC          = 0x2000
Scaled offset           = 4 * 4 = 0x10
Effective address       = 0x2010

v0 = Memory32[0x2010]
```

A PC-relative `LW` is especially relevant for **literal pools**: code may load a nearby constant or a pointer from data embedded near instructions, then use that pointer as a second-stage base.

```
0x2002    LW v0, 0x10(pc)       # reads memory at 0x2010
0x2004    ...
0x2010    00 FA 00 40          # 32-bit data value = 0x4000FA00

After load: v0 = 0x4000FA00
```

The **bytes `00 FA 00 40` are data**, not another instruction merely because they live nearby in the same firmware file. A wrong disassembly boundary can accidentally decode a literal pool as code.

Technical exception: The exact PC-relative BasePC changes when the instruction occurs in certain **jump delay slots**; for extended versions, the base is the EXTEND instruction address. [M1, Table 3.14]. The simple example above explicitly excludes a jump delay slot.

7.F. EXTEND: two halfwords, one instruction

Rule: EXTEND enlarges the immediate/displacement capability of an eligible MIPS16e instruction by combining two halfwords into one effective **4-byte instruction**. [M1, §§3.11, 3.14]

```
PC = 0x4000

Memory bytes:    [ first 16-bit halfword ] [ second 16-bit halfword ]
Interpretation:  [ EXTEND prefix          ][ eligible instruction     ]
Combined length: 4 bytes
Next boundary:   0x4004
```

An extended instruction is **not** 'first execute EXTEND, then execute an ordinary instruction' in the usual sequential sense. Decode the pair together to determine the full immediate and instruction semantics.

Common failure: Search for a value such as `0x204` and assume it must appear as a conventional MIPS32 16-bit I-type immediate. In compressed/extended MIPS16e instructions, the bits for the logical immediate can be **split across the two halfwords** or scaled, depending on instruction format. A linear byte search may fail even though the program accesses that offset.

Length matters: In 16-bit instruction streams, a sequence of lengths `2, 2, 4, 2` advances as `A, A+2, A+4, A+8`. A blanket `PC += 2` or `PC += 4` is wrong.

Caution on instructions: Although the target is described as MIPS16e, some newer descriptions/documentation mention **MIPS16e2**. Do not assume MIPS16e2-specific instructions are supported by your unknown proces-

sor just because a modern Ghidra source tree has decoders for them. [G3]

7.G. Decode a classic MIPS32 instruction for comparison only

Rule: Conventional MIPS32 I-type format is useful to read, but its 6/5/5/16-bit fields do **not** define the general MIPS16e decoder.

```
# CLASSIC MIPS32 mode ONLY
lw $v0, 0x204($s0)
```

```
Field       Bits                Meaning
opcode      100011              LW (0x23)
rs          10000               S0 (register 16)
rt          00010               V0 (register 2)
immediate   0000001000000100    0x0204

whole word = 0x8E020204
little endian instruction bytes = 04 02 02 8E
```

And `sw $v0, 0x204($s0)` is:

```
opcode = 101011 (0x2B)
word = 0xAE020204
little endian instruction bytes = 04 02 02 AE
```

Do not confuse machine code with data: The `SW` instruction word is *not* what gets stored. The CPU stores the **32-bit runtime value of `$v0`** at the calculated effective address. If `$v0=0x12345678`, those bytes are `78 56 34 12` in little endian.

Reverse engineering consequence: A match for the byte pattern `04 02 02 AE` only supports a **classic MIPS32 `sw` hypothesis after ISA mode and alignment have been verified**. Treating an arbitrary four-byte window in MIPS16e code as a classic word can invent an instruction that is not actually executed.

7.H. Ghidra mode selection: decode before decompile

Rule: Ghidra needs the correct **program language, ISA-mode context, and instruction start addresses** to decode mixed MIPS16e code reliably.

Your established language is **`MIPS:LE:32`**. Ghidra's MIPS implementation has an **`ISA_MODE` processor context**; in its MIPS16e-capable disassembly command, value **1** enables MIPS16e and value **0** selects ordinary MIPS disassembly. The command uses **2-byte alignment** for MIPS16e starts and **4-byte alignment** for ordinary MIPS32 starts. [G1, G2]

A safe inspection sequence

1. **Locate the bytes** in the Listing/Bytes view and note their exact address and memory block.
2. **Confirm the language** and endianness. Do not silently reinterpret an imported raw binary as an ELF with inferred metadata.
3. **Inspect `ISA_MODE`** at a known entry point; if wrong, use the processor-context/register-management tools supported by your installed Ghidra version to set the desired context before disassembling.
4. **Clear incorrect disassembly only in the relevant verified range** and re-disassemble with a known correct entry, rather than mass-clearing large regions.
5. **Verify instruction sizes**, branch destinations, and any EXTEND pair against the actual bytes.
6. **Follow calls and indirect returns** to check whether their destinations switch between MIPS16e and classic MIPS32.
7. **Only after that**, consult the Decompiler and Function Graph.

UI caveat: Exact menu names and availability differ by Ghidra version. Ghidra's official help documents processor options and register-context control; its open-source `MipsDisassembleCommand` explicitly describes the `ISA_MODE` behavior. [G1, G2, G4]

Signals that your decoder mode may be wrong

- Functions begin at unexpected halfword boundaries.
- The output contains many unexpected illegal/undefined instructions.
- Branches land inside the middle of another disassembled instruction.
- A 4-byte EXTEND sequence has been split into independent instructions.
- Cross-mode calls lead to a nonsensical function body.
- Decompiled code contains implausible operations or missing control-flow edges despite meaningful raw bytes.

None of these alone proves wrong mode. Corruption, mixed data/code, wrong base address, unsupported extensions, and genuine unusual compiler output can also produce strange listings.

7.I. Functions, symbols, and XREFs: what each one can prove

Rule: An **XREF** is a known static reference; it is not proof that the reference executes or that no other dynamic use exists.

Ghidra tool	What it helps answer	Limitation
Listing	What bytes currently decode to	Wrong mode or start yields misleading instructions
Bytes view	Exactly which bytes exist	Bytes alone do not name data types
Decompiler	Plausible C-like behavior	Types and flow can be inferred incorrectly
Function Graph	Branch and basic-block structure	Needs correct instruction decoding and boundaries
XREFs	Who statically references a location	Indirect references may be absent
Symbol Tree	Named functions/data/symbols	Names may be autogenerated or inaccurate
Memory Map	Mapped blocks, base addresses, permissions	Raw images may not reflect runtime mappings
Data Type Manager	Candidate struct layouts	Assigned types are hypotheses until corroborated
Register context	Per-address processor state, including mode	Mode propagation depends on flow and tooling

Example: A static `jal` to `FUN_80001234` is evidence of a call edge, but a callback loaded from a table and invoked indirectly might produce no direct call XREF to that function. A renamed variable such as `status_flags` remains a user-imposed model, not suddenly ground truth.

7.J. Backward slicing: find the actual pointer base

Rule: The most important question about `lw v0, 0x204(s0)` is often **where `$s0` came from**, not how to add `0x204`.

A backward slice is the sequence of earlier operations that could have produced the value in a register at a particular instruction.

`Schematic code, representation-agnostic:`

```
function Reader(a0):
    s0 = a0
    v0 = MEM32[s0 + 0x204]
    return v0
```

```
caller:
    a0 = 0x4000FA00
    call Reader
```

Now we can conclude that, **for that observed call with those register values**, the read uses effective address `0x4000FC04`. We still cannot infer the semantic type of the object at `$s0` from that access alone.

A provenance checklist

- **Last reaching definition:** At the load, which earlier instruction most recently wrote `$s0`?
- **Copies:** Was it copied from `$a0`, `$v0`, `$gp`, or another register?
- **Loads:** Did that predecessor come from global memory or a literal pool?
- **Computed bases:** Was a constant added? Is the pointer the start of a subregion or a middle field?
- **Callers:** If it came from `$a0`, what did each caller pass?
- **Alternate paths:** Can a branch assign different pointers to `$s0` before the load?
- **Actual execution:** Which code path ran during the event of interest, if known?

Crucial: A slice may have **multiple possible definitions** because execution can reach the load from different branches. If one caller passes `0x4000FA00` and another passes `0x5000FA00`, you do not have a single universal base value.

7.K. Trace `base + 0x204` without inventing a struct

The original textbook discussed a candidate word at an offset relative to a shared-memory region. Use this as a **worked arithmetic example**, not as proof of any particular firmware function's purpose.

```
Suppose the value actually held in s0 is 0x4000FA00.
The instruction accesses an offset of 0x204.

  0x4000FA00
+ 0x00000204
------------
  0x4000FC04
```

Two unrelated questions must be answered separately:

1. **Address identity:** Does this pointer truly refer to the intended shared-memory table in the modem's address view?
2. **Semantic identity:** Does the word at `+0x204` represent the suspected state/flag/control field?

One **matching numeric offset is not proof**. Many structures and arrays use offset `0x204`, and the base can change across callers or runtime configurations.

Observed / inferred / unproved worksheet

Statement	Status with the preceding example only
Instruction reads 4 bytes from `s0 + 0x204`	Supported if decoder, ISA mode and instruction semantics were verified
For `s0=0x4000FA00`, address is `0x4000FC04`	Mathematically certain
The read belongs to a particular named struct	Not established
The word is a screen-state flag	Not established
This reader executes exactly when the screen changes	Not established
There is a writer to this same backing word elsewhere	Not established

Address-space caveat: Firmware may be analyzed from a modem address view while an AP observes the same backing region at another virtual or physical address. You must know the actual mapping before claiming two differently numbered addresses refer to the same storage.

7.L. Two-step indirection: pointer stored near the PC

Rule: Seeing a constant load followed by a structure-field load can mean the code is first **fetching a pointer** and then **following it**.

```
0x2002: LW v0, 0x10(pc)       # MIPS16e example from 7.E
0x2010: 00 FA 00 40          # data = pointer 0x4000FA00

Later:
    s0 = v0
    v1 = MEM32[s0 + 0x204]
```

Step by step:

1. The instruction at `0x2002` loads a **word from nearby memory** into `$v0`.
2. That word's numeric value is `0x4000FA00`.
3. A later instruction treats that number as a **pointer**.
4. The later load reads from `0x4000FC04`.

The value stored at `0x2010` is not the target field. It is an address pointing toward the memory region where the field might be located.

This is why **the same bytes can represent an integer, pointer, flag, or code** depending on actual use. Data-type analysis relies on *how* values participate in computation and memory operations.

7.M. Search strategies beyond one literal immediate

Rule: To find uses of a field, search for **data flow and access patterns**, not only textual occurrences of a constant.

Useful approaches:

- **Known address bytes:** Search memory for encoded pointers and constants, but account for little-endian ordering and relocation.
- **Scalar searches:** Search for `0x204`, while recognizing it may be split, scaled, or computed.
- **Base provenance:** Trace a known pointer from a literal pool, global lookup, parameter, or initialization routine.
- **Neighboring accesses:** Search for repeated uses of nearby offsets (such as 0, 4, 8, 0x204) with a common base register.
- **Call graph:** Identify readers/writers reached from relevant dispatchers; use indirect call tables where needed.
- **XREFs:** Inspect direct references; independently account for references missing from the static graph.
- **p-code:** Use lower-level semantics for LOAD, STORE, conditional branch, and register propagation if the decompiler hides source details. [G5]

Why a raw-byte search may miss it: In classic MIPS32, `0x204` might appear as a signed immediate field. In MIPS16e, a compact encoding can represent an offset in **scaled units** or use EXTEND to spread fields across halfwords. The compiler can also compute `base+0x200` and then use +4 instead of encoding a single +`0x204` access.

7.N. Structures, arrays, and memory-mapped registers

Rule: The same load instruction can correspond to **multiple plausible C data types**.

```c
// Hypothesis A: struct
struct Status { int mode; int flags; };
```

```
result = ptr->flags;            // +4

// Hypothesis B: array
result = numbers[1];           // +4 for 32-bit int

// Hypothesis C: raw buffer
result = *(uint32_t *)(buf + 4); // +4

// Hypothesis D: memory-mapped register block
result = *(volatile uint32_t *)(mmio_base + 4); // +4
```

They could all lead to a similar word load. To distinguish them, analyze **how the pointer is obtained**, whether accesses occur in loop patterns, whether values are used as pointers or bit masks, whether the region is memory-mapped I/O, and whether matching symbols or structures exist.

Do not overfit a decompiler's type suggestions. For example, `undefined4 *param_1` means the decompiler has established a 4-byte pointer-like access pattern, not a source-level variable name.

7.O. P-code: why it helps when assembly syntax varies

Rule: Ghidra's p-code describes instruction effects in a lower-level intermediate representation, helping compare **what code does** without relying entirely on a mnemonic's textual form. [G5]

A load can be reasoned about as:

```
addr   = base + offset
value  = LOAD(memory_space, addr)
output = value
```

A store:

```
addr = base + offset
STORE(memory_space, addr, input)
```

A branch:

```
if condition then goto target else goto fallthrough
```

This helps when compact instructions have unusual syntax or implicit operands. However, **p-code quality depends on the selected processor language and correct mode**. If the raw bytes were misdecoded, the p-code models the wrong operation.

7.P. A practical Ghidra investigation session

Case: A suspicious reader of a shared-memory field, starting from a candidate function rather than a known live trace.

Pass 1: Establish instruction truth

- Record function entry, current ISA mode, and the literal bytes around the load.
- Confirm whether each instruction is 2 or 4 bytes.
- Verify branch targets and exclude mistaken data-as-code regions.

Pass 2: Build the data-flow hypothesis

- Find the reaching definition of `$s0`.
- If `$s0` comes from `$a0`, list the caller arguments.
- If `$s0` comes from a global lookup, identify the table/literal and follow the pointer.
- For each candidate, calculate the **actual effective address**.

Pass 3: Establish semantic evidence

- Inspect neighboring fields and bitmask operations.

- Look for source-derived names, type information, or independently observed state changes.
- Check whether the code **reads** or **writes** the field and whether it is behind a conditional branch.

Pass 4: Write down confidence, not just conclusions

```
Observation: verified MIPS16e load from s0 + 0x204.
Evidence:    <instruction address, raw bytes, length, mode>.
Provenance:  caller sets s0 to known base on one observed path.
Address:     0x4000FC04 on that path.
Role:        candidate status-field reader, not yet confirmed.
Missing:     runtime execution evidence or independent type proof.
```

This style of evidence ledger prevents accidental promotion of a plausible guess into a fact.

7.Q. A short Python decoder for two verified MIPS16e branch forms

Rule: Start with a *small, auditable decoder for known forms*, not a universal decoder invented from a few examples.

The following educational script decodes only **nonextended MIPS16e BEQZ and BNEZ**. It verifies the 5-bit opcode, translates the 3-bit compact register field, reads a signed 8-bit offset, and computes the branch target. It rejects other instructions on purpose.

```python
COMPACT_REGS = ("s0", "s1", "v0", "v1", "a0", "a1", "a2", "a3")

def sign8(x):
    return x - 256 if (x & 0x80) else x

def decode_branch(raw_2_bytes, pc):
    if len(raw_2_bytes) != 2:
        raise ValueError("Expected exactly one 16-bit instruction")
    hw = int.from_bytes(raw_2_bytes, "little")
    opcode = (hw >> 11) & 0x1F
    if opcode not in (4, 5):
        raise ValueError("Not short BEQZ/BNEZ; check EXTEND/mode")
    compact_reg = (hw >> 8) & 7
    signed_offset = sign8(hw & 0xFF)
    target = (pc + 2 + 2 * signed_offset) & 0xFFFFFFFF
    mnemonic = "BEQZ" if opcode == 4 else "BNEZ"
    return mnemonic, COMPACT_REGS[compact_reg], signed_offset, target

print(decode_branch(bytes.fromhex("03 22"), 0x2000))
# ('BEQZ', 'v0', 3, 8200)   8200 decimal = 0x2008

print(decode_branch(bytes.fromhex("FC 2A"), 0x2100))
# ('BNEZ', 'v0', -4, 8442)  8442 decimal = 0x20FA
```

Guardrails: This script does **not** verify whether a given address is actually MIPS16e code or inside a larger instruction. Only call it after mode and instruction-boundary checks. It does not handle EXTEND, JAL/JALX, unusual modes, or anything except these two forms.

Why such a narrow script is valuable

You can compare one raw halfword against the official manual, your script, and Ghidra. If all three agree on the register, sign, and target address, confidence increases. Then extend the decoder one **documented format** at a time, each with regression tests.

7.R. Anti-patterns: eight ways to fool yourself

1. **Assume 4 bytes per instruction:** loses MIPS16e boundaries.
2. **Decode any four bytes as a MIPS32 word:** may turn compact code into false `lw`/`sw` results.

3. **Ignore ISA mode tags on indirect targets:** causes mode confusion.
4. **Treat BEQZ as having a classic delay slot:** makes path tracing wrong.
5. **Call every positive offset a struct field:** confuses arrays and register blocks.
6. **Assign semantic meaning from one bitmask:** `andi ...,4` proves a bit test, not a named state.
7. **Treat Ghidra XREF absence as non-use:** dynamic addressing may hide references.
8. **Mistake AP and modem numeric addresses for the same address view:** mapping requires proof.

7.S. Chapter 7 mastery problems

1. Decode little-endian bytes `03 22` at PC `0x2000` in verified MIPS16e mode. Give mnemonic, register, and target.
2. Decode `FC 2A` at PC `0x2100` in verified MIPS16e mode. If V0 is nonzero, where next?
3. Decode `04 B2` at PC `0x2002` in verified MIPS16e mode, not in a jump delay slot. Give the effective memory address.
4. Why are bytes `04 02 02 8E` not by themselves evidence of a classic MIPS32 `LW` in a MIPS16e stream?
5. A compressed decoder reports instruction lengths `[2,4,2,2]` starting at `0x3000`. List the four starting addresses and the next free boundary.
6. A suspected `LW` accesses `s0+0x204`. Name **two** distinct types of objects that could produce this access.
7. You find a tagged function pointer `0x80001201` in a MIPS16e-capable environment. What fetch address and ISA mode can it indicate when used by a suitable indirect jump?
8. A decompiler reports `undefined4 *param_1`. Does that prove the original variable was a pointer to a particular named struct?
9. A branch at the start of the function seems to jump into an `EXTEND` pair's second halfword. What is the first category of error to investigate?
10. Why does a data-flow slice need to consider **more than one predecessor path** to a load?

Answers follow in Appendix B; don't consult them until you have written your reasoning.

APPLIED CASE STUDIES: Put Chapters 5-7 together

Case 1. Recover a branch from two bytes

Goal: Determine the control-flow target using bytes, mode, and register state.

You are told that the code entry at `0x6200` is **MIPS16e**, and memory begins with:

```
Address   Bytes
0x6200    03 22
0x6202    11 00      # unrelated illustrative raw bytes
0x6204    22 00
0x6206    33 00
0x6208    44 00
```

Step 1: Convert the halfword. `03 22` in little endian means `0x2203`.

Step 2: Split fields. `00100 | 010 | 00000011`, giving BEQZ, register `$v0`, offset +3.

Step 3: Compute target. `0x6200+2+(3<<1)=0x6208`.

Step 4: Apply predicate. If `$v0=0`, next instruction starts `0x6208`. If `$v0=7`, fall through to `0x6202`.

What we cannot infer: What condition `$v0` represents. A compiler could be checking a pointer, boolean, integer result, or error code. The semantic name needs more context.

Independent variant

Repeat with bytes `FE 22` at `0x6200`. What is the signed displacement? Which earlier address is targeted when `$v0=0`? What is the fall-through address when `$v0=1`?

Case 2. Why MIPS16e and classic MIPS32 have different traces

Goal: Compare two superficially similar branch sequences and avoid a false fall-through.

```
Scenario A: MIPS16e nonextended BEQZ at 0x7000 (length 2)
     0x7000   BEQZ v0, 0x7008
     0x7002   increment s0
     ...
     0x7008   next block

Scenario B: classic MIPS32 BEQ at 0x7000 (length 4)
     0x7000   BEQ v0, zero, 0x7010
     0x7004   increment s0
     ...
     0x7010   next block
```

Assume `$v0=0`, `$s0=10`. In **A**, `$s0` remains 10 before reaching `0x7008`, because the branch has **no delay slot**. In **B**, `$s0` becomes 11 before reaching `0x7010`, because `0x7004` is a classic delayed branch slot.

Lesson: Seeing a familiar mnemonic-like branch is insufficient. Record the mode, width, and delay behavior or you'll predict different runtime state.

Independent variant

Change `$v0=9` in both scenarios. Which increment executes? Which target becomes unreachable on this pass?

Case 3. Reconstruct a stack frame before naming locals

Goal: Use actual arithmetic to map saved state, rather than guessing variable names.

Suppose entry SP is `0x80001000`. A conventional MIPS32 function shows:

```
addiu $sp, $sp, -0x20
sw    $ra, 0x1C($sp)
sw    $s0, 0x18($sp)
sw    $a0, 0x10($sp)
...
lw    $s0, 0x18($sp)
lw    $ra, 0x1C($sp)
addiu $sp, $sp, 0x20
jr    $ra
nop
```

Step 1: New SP = `0x80000FE0`.

Step 2: Saved RA at `0x80000FFC`; saved S0 at `0x80000FF8`; the saved copy of A0 at `0x80000FF0`.

Step 3: At exit, S0 and RA are restored from memory and SP returns to `0x80001000`.

Interpretation: The register `$a0` may be saved because its input value is needed later, not because all arguments are callee-saved. Saving an argument on the stack is not evidence of a particular source-language object type.

Independent variant

If the frame were `0x30` bytes instead, but the saved offsets remained the same, calculate the new absolute addresses. Why is it dangerous to assume a slot always lies at a fixed absolute address?

Case 4. A nested call overwrites the return link

Goal: See why nonleaf functions often save `$ra`.

```
A continuation after calling B: 0x1008
B continuation after calling C: 0x2040
```

Assume the appropriate calls place their continuation addresses into `$ra` according to the mode-specific link rules. After A calls B, `$ra` refers to **A's continuation**. When B calls C, the new call puts **B's continuation** in `$ra`.

Therefore B must preserve the original link to A somewhere if it plans to return to A afterward. It can save it to a stack slot, restore it after C returns, and finally return to A. Without that, a return could go back to B's continuation again instead of back to A.

Advanced subtlety: The saved link can include a **low-bit ISA-mode tag**. For mixed-mode code, losing that bit may restore the right-looking numeric address but the wrong decoder mode.

Independent variant

B uses a `$s0` register and also calls C. Which two values may B have to preserve, and for different reasons?

Case 5. Trace a PC-relative pointer into a structure-like field

Goal: Separate raw data, pointer value, effective address, and field semantics.

Verified MIPS16e code at `0x2002` begins with `04 B2` (our LWPC example).

```
0x2002: bytes 04 B2      -> PC-relative LW v0, offset 0x10
0x2010: bytes 00 FA 00 40 -> data word 0x4000FA00
```

Step 1: PC-relative load reads four bytes from `0x2010` and writes `0x4000FA00` into `$v0`.

Step 2: A later operation copies `$v0` into `$s0`.

Step 3: A later verified load reads 32 bits from `$s0+0x204`, so effective address = `0x4000FC04`.

Step 4: If the word is `0x0000000C`, masking with `0x04` gives `0x04` (bit 2 is present).

What is proven: The two addresses and the masked result given the stipulated execution state.

What is unproved: The real semantic name of the field, how its value is initialized, whether the code ran during any screen transition, and whether another address view maps to the same physical storage.

Independent variant

If the literal at `0x2010` changes to bytes `00 00 10 40`, what pointer is loaded? What address does an access at `+0x204` reach?

Case 6. An XREF you do not see

Goal: Explain why an indirect call may escape a naive static search.

```
callback = callbacks[index];
result = callback(5);
```

At machine level, code can calculate an index, load a function pointer from the table, place it in a register, set up `$a0=5`, and perform an indirect jump-and-link. **The direct encoded instruction does not contain the eventual callee address.**

Ghidra may show an XREF to the **table** but no direct call XREF to every target function, especially if table entries are relocated or filled dynamically.

A defensible strategy is to recover the table, assess entry types and mode tags, identify the index range, and correlate observed control-flow targets. An XREF to `callbacks` is evidence about data access, not necessarily a complete set of destinations.

Independent variant

Why might the decompiler show `(*functionPtr)(5)` instead of a named function call, even if the executed target is always the same at runtime?

Case 7. A mixed-mode call and an apparently odd return pointer

Goal: Distinguish an ISA tag from an unaligned instruction fetch.

```
0x7000: MIPS16e JALX helper     [4 bytes]
0x7004: valid delay-slot instr [2 bytes]
0x7006: continuation           [2 bytes]

helper: classic MIPS32 code
```

For this MIPS16e JALX example, `$ra` receives `0x7007`: the **continuation address `0x7006` tagged with bit 0 = 1**. The target function runs in **classic MIPS32** mode. A suitable indirect return uses the saved tag to restore **MIPS16e** mode when execution returns to `0x7006`. [M1, JALX pp. 86-87, §3.7]

Key implication: A debugger display that naively labels `0x7007` as a literal instruction-fetch location is misleading. The fetch address is aligned as required; the low bit is a control-state tag.

Independent variant

What would be the consequence of clearing bit 0 of the saved return pointer before returning, without checking how mode restoration works?

Case 8. A 4-byte instruction is mistaken for two instructions

Goal: Learn why a disassembly can drift out of synchronization.

The bytes at a verified MIPS16e entry contain an EXTEND prefix followed by its eligible second halfword. At the next memory halfword address, a mistaken decoder starts a new instruction.

```
Correct:
0x3000: [EXTEND halfword][second halfword] = ONE instruction, size 4
0x3004: [next instruction]

Incorrect:
0x3000: pretend prefix is a 2-byte standalone instruction
0x3002: decode the second half as if it begins a new instruction
0x3004: ...
```

A false branch target, broken function boundary, or nonsensical decompiler variable can follow from **one boundary error**.

Diagnostic action: Inspect the raw halfwords and reapply the manual's format rule for the prefix before trusting the rest of the listing.

Independent variant

If a decoder incorrectly consumes 2 bytes for a 4-byte instruction at `0x3000`, at which address does its next decode begin, and why is that address invalid as a separate instruction start in this case?

Case 9. One access, four explanations

Goal: Avoid premature structure claims.

You find a verified load from `base + 4`. Four candidate original expressions are:

```
status->flags
numbers[1]
*(uint32_t *)(raw + 4)
*(volatile uint32_t *)(mmio + 4)
```

All may produce an equivalent address calculation and 4-byte load. To discriminate them:

- Observe whether the base came from a struct pointer argument, array allocation, raw buffer, or MMIO mapping.
- Look for more accesses: repeated stride-4 indexed reads suggest an array; stable offsets used in different ways suggest a record or register block.
- Examine volatility and side effects, but be cautious because optimizations and disassembly representations may hide source-level qualifiers.
- Use available symbols, debug data, matching source, or independently confirmed hardware memory maps.

Strong conclusion: Without more evidence, call it **a word load at base+4**. Do not confidently label it `status->flags`.

Independent variant

If the next instruction performs `andi ..., 0x08`, does that *prove* the loaded word is a flags field? Give a precise statement of what it establishes.

Case 10. Write a defensible research note

Goal: Convert analysis into repeatable evidence instead of a vague 'looks like' claim.

Poor note: "Found the state struct. `lw +0x204` is definitely the screen variable."

Better note: "At address `0x____`, in verified MIPS16e mode, the Listing decodes a 4-byte load from `base+0x204`. The nearest reaching definition of base comes from argument register A0. Caller at `0x____` constructs/passes `0x4000FA00` on one path, implying address `0x4000FC04` for that path. Other callers and runtime execution are not yet established. The semantic field name remains a hypothesis."

The **better note** records the code location, the mode, the provenance, the arithmetic, the scope of the evidence, and the remaining uncertainty. A reverse-engineering report should remain correct even if later analysis disproves the initial semantic guess.

Independent variant

Rewrite the following claim as two separate statements, one fact and one hypothesis: “This indirect load proves the modem changes a screen-state flag every time it sleeps.”

APPENDIX A. Fast reference cards

A.1. Control-flow card

Encounter	Ask first	Correct quick rule
`BEQZ rx,target` in MIPS16e	Is rx zero?	Nonextended form: PC+2+(signed imm8*2), no delay slot
`BNEZ rx,target` in MIPS16e	Is rx nonzero?	Nonextended form: PC+2+(signed imm8*2), no delay slot
BTEQZ in MIPS16e	Is T (`$t8`) zero?	Branch if T zero; no delay slot
JAL in MIPS16e	What is slot length?	Delayed call; link address includes mode tag
JALX in MIPS16e	What mode is callee?	Delayed call that exchanges ISA mode
JR in MIPS16e	What does source register hold?	Register jump with a delay slot
JRC in MIPS16e	What does source register hold?	Register jump without a delay slot
Classic MIPS32 BEQ	Mode/revision?	Conventional pre-R6 model uses delay slot and word-scaled displacement

MIPS16e mode and conventional MIPS32 mode share 32-bit GPRs. Instruction width is not register width.

A.2. Function/stack card

Register	Typical O32 purpose	Do not assume
`$a0-$a3`	Initial integer/pointer argument words	They necessarily contain source-level scalar integers
`$v0-$v1`	Return values	Every return is meaningful in V0 or that every function returns
`$t0-$t9`	Caller-saved temporaries	Callee will preserve them
`$s0-$s7`	Callee-saved values	They specifically hold C ‘saved’ objects
`$sp`	Stack pointer	A numeric SP offset identifies the same absolute address across SP changes
`$fp/$s8`	Optional stable frame base	Every function uses a frame pointer
`$ra`	Link/return destination	It survives nested calls without preservation
`$gp`	Global/static/GOT base in common conventions	It inherently represents C scope
`$t9`	PIC call target in some conventions	Every indirect call through T9 is PIC/O32

A.3. Byte/decode card

```
Target: MIPS32 little endian; code under investigation: MIPS16e

16-bit halfword 0x2203 -> bytes 03 22
```

```
32-bit classic MIPS32 LW word 0x8E020204 -> bytes 04 02 02 8E
MIPS16e short BEQZ encoding:
   00100 [rx 3 bits] [signed offset 8 bits]
MIPS16e short BNEZ encoding:
   00101 [rx 3 bits] [signed offset 8 bits]
MIPS16e PC-relative LW:
   10110 [rx 3 bits] [unsigned offset 8 bits]
   effective address = align_down(BasePC,4) + 4*offset8
   [ordinary case outside certain jump delay slots]
MIPS16e compact register map:
   000 s0 | 001 s1 | 010 v0 | 011 v1
   100 a0 | 101 a1 | 110 a2 | 111 a3
```

DO NOT use the abbreviated formula for an EXTEND pair. Its immediate decoding depends on the documented extended instruction form. Always look up the exact form in [M1].

A.4. Thirty-second Ghidra checklist

```
[ ] Correct imported bytes / memory base
[ ] Language MIPS:LE:32 (established target)
[ ] Correct per-address ISA_MODE (MIPS16e or MIPS32)
[ ] Correct 2/4-byte instruction boundaries
[ ] EXTEND pair not split; data not mistaken for code
[ ] Branch destination recalculated independently
[ ] Delay-slot rule matched exact instruction
[ ] RA / mode-tag behavior considered at calls and returns
[ ] Pointer base traced to earlier definitions and callers
[ ] Semantic type/field claims graded as hypotheses or evidence
```

APPENDIX B. Worked answers and diagnostic explanations

How to use this appendix: Read one problem, write your own answer on paper, then compare your *reasoning* rather than only the final number. If the number matches for the wrong reason, redo the exercise.

B.1. Chapter 5 checkpoints

5.A. The branch is taken because **`$v0` contains zero**. The branch target's numeric value does not participate in the equality test.

5.B. A taken MIPS16e BEQZ has **no delay slot**, so the instruction following it is skipped on the taken path. MIPS16e JAL has **one instruction delay slot**, which runs before control reaches the callee; the instruction must be legal in that position.

5.C. Base = `0x1200+2=0x1202`; displacement `-3 * 2 = -6`; target = **`0x11FC`**. When the register is nonzero, BEQZ is not taken; fall-through starts at **`0x1202`**. The displacement can point to a lower address; that does not change the condition.

5.D. If T is `0x00000004`, BTEQZ **does not** branch because T is nonzero. BTNEZ **does** branch. It is the zero/nonzero property of T, not the value of the target, that controls the condition.

5.G. `AND with 0x04` preserves **bit 2** of a register; BNEZ can test whether the masked result is nonzero. It suggests a flag test, but **does not prove the name, origin, or meaning** of that bit, nor that the branch was executed during a particular event.

5.J. If initial V0 is `0x08`, `0x08 & 0x04 = 0`. Thus BEQZ is **taken** to `0x500A`, adds 8, and continues at `0x500C`. Final S0 = **`0x4000FA08`**. The instruction at `0x5004` is skipped; this is not a classic delayed branch.

B.2. Chapter 6 checkpoints and mastery

6.E. Initial SP `0x80001000`; subtract `0x20` (32 decimal) gives **`0x80000FE0`**. Add slot offset `0x1C` (28) to obtain **`0x80000FFC`**. That is the address where `sw ra,0x1C(sp)` stores the RA value.

6.G. A function may save RA because **its own nested call overwrites the link register**. If it loses the original destination, it may be unable to return to its caller correctly. The exact link value and mode tag depend on instruction mode and call form.

6.I. `$gp` is **not** a hardware-enforced 'global scope' pointer. `$t9` being used in a PIC-like sequence is suggestive but does **not prove** the whole firmware follows O32 or that the relevant code is PIC. Inspect entry sequences, calls, global accesses, and metadata.

Mastery 1. B, the **callee**, must restore S0 to **`0x1234`** before returning under O32 if B modifies it. It can save the original on its stack and reload it; A may rely on the ABI contract.

Mastery 2. No, if A never again needs T0, it need not preserve that old value. Caller-saved means **the caller decides based on liveness**, not that every caller always stores all T registers.

Mastery 3. B's call to C installs a **new return link**. B needs its incoming link back to A if it is to return to A. Saving RA on the stack is common.

Mastery 4. `0x9000-0x30 = 0x8FD0`. Then `0x8FD0+0x2C` = **`0x8FFC`**. If the starting SP changes, the absolute address changes even if the relative frame offset remains 0x2C.

Mastery 5. `$a0=0x1000` contains an **address**; the memory content at that address is `0xDEADBEEF`. Passing a pointer and passing the pointed-to value are different operations.

Mastery 6. No. `jalr $t9` alone is not enough. Look for consistent PIC linkage conventions, GOT-relative loads, GP construction, relocation information, and call stubs. The compiler can also use a register-indirect call for ordinary function pointers.

Mastery 7. No. A leaf function can return without allocating stack memory. Optimizers may omit a frame pointer or omit a frame altogether. Some MIPS16e functions can use compressed frame instructions.

Mastery 8. MIPS16e **SAVE** can perform register saves and frame setup compactly, so a prologue need not contain a visibly separate `sw ra,...(sp)` pair. Decode the actual instructions rather than pattern-matching

only classic MIPS32 prologues.

B.3. Chapter 7 checkpoints and mastery

7.C. 0x2102 in binary is 00100 001 00000010: **BEQZ**, compact register 001 = **S1**, displacement **+2 halfwords**. At 0x6000, target = 0x6000+2+4 = **0x6006**.

Mastery 1. 03 22 = halfword 0x2203 = BEQZ V0, +3 halfwords. Target at 0x2000 is **0x2008**; fall-through is 0x2002.

Mastery 2. FC 2A = halfword 0x2AFC = BNEZ V0, -4 halfwords. If V0 nonzero, target at PC 0x2100 is **0x20FA**.

Mastery 3. 04 B2 = 0xB204 = a nonextended MIPS16e PC-relative LW into V0. At 0x2002, align BasePC down to 0x2000; scale immediate 4 by 4 = 0x10; effective address = **0x2010**. This excludes unusual delay-slot BasePC cases.

Mastery 4. Those bytes do encode a classic MIPS32 LW *if* decoded as an aligned classic MIPS32 instruction. But **bytes are not intrinsically classic MIPS32**. In MIPS16e mode, they are decoded as sequential halfwords, possibly with different meanings, starting from actual instruction boundaries.

Mastery 5. Instruction starts: **0x3000, 0x3002, 0x3006, 0x3008**. Next free boundary: **0x300A**. Add each *actual* instruction length to obtain the next start; do not assume a constant stride.

Mastery 6. A named struct field, an array element, a raw buffer word, or a memory-mapped register could all appear at base+0x204. One instruction does not establish which.

Mastery 7. The tagged value **0x80001201** can indicate **MIPS16e mode (bit 0 = 1)** and fetch address **0x80001200** when interpreted by a suitable mode-aware register-indirect jump. The low bit does not mean fetching a halfword at an odd byte address.

Mastery 8. No. It suggests decompiler inference of a 4-byte pointer-like data access, but not the original variable name or semantic type. Validate against other accesses and independent evidence.

Mastery 9. First inspect **ISA mode, instruction start, length, and possible EXTEND splitting**. A false target into the middle of an instruction can result from wrong disassembly boundaries, wrong branch scaling, or wrong mode. Do not patch the target before checking decode correctness.

Mastery 10. Different predecessor blocks or different callers can assign **different base values**. A valid slice must account for alternate reaching definitions and conditions; otherwise a single convenient path may be mistaken for the only behavior.

B.4. Independent variants from the ten case studies

Case 1 variant. FE 22 = halfword 0x22FE; opcode 00100 = BEQZ, register 010 = V0, offset 0xFE = **-2**. At PC 0x6200, target = 0x6202-4 = **0x61FE** if V0 is zero. If V0 is 1, not taken; next is **0x6202**.

Case 2 variant. With V0=9, the conditional branches are **not taken** in both scenarios. The increment at 0x7002 in MIPS16e then executes as ordinary fall-through, and the increment at 0x7004 in classic MIPS32 executes as the delay slot. The designated taken target is not visited as the immediate branch destination on that pass (it might be reached later by unrelated paths, which the abbreviated example does not exclude).

Case 3 variant. Entry 0x80001000 minus 0x30 = 0x80000FD0. Offset 0x1C → **0x80000FEC**; offset 0x18 → **0x80000FE8**; offset 0x10 → **0x80000FE0**. Offsets describe locations **relative to the current SP**, not permanent absolute addresses.

Case 4 variant. B may have to preserve **incoming RA** because it calls C, and **incoming S0** because O32 expects callee preservation if B modifies S0. They are independent obligations: one protects the return destination, the other protects a nonvolatile general-purpose register.

Case 5 variant. 00 00 10 40 is **0x40100000** in little endian. Plus 0x204 gives **0x40100204**. The result is address arithmetic, not proof of a source type.

Case 6 variant. A static analyzer may know only that a register contains a function pointer at that call site, not its runtime value. Indirect calls are not encoded with a direct address operand and may depend on table

contents/index values; therefore the decompiler can reasonably retain indirect-call syntax.

Case 7 variant. Clearing bit 0 can change an indirect return from **MIPS16e mode** to **classic MIPS32 mode**. The same aligned bytes may then be decoded incorrectly and execution can fault or behave unpredictably. Preserve ISA-mode information unless the architecture-defined transition calls for a change.

Case 8 variant. The bad decoder begins at **`0x3002`**, but that is the **second halfword of the same 4-byte MIPS16e instruction**, not the beginning of an independent instruction in this case. Re-check the prefix and recompute the next boundary `0x3004`.

Case 9 variant. `andi ...,0x08` establishes a **bit-3 test/selection** on the loaded value, after whatever preceding transformations occurred. It **supports** a flags/status hypothesis but does not prove the field's semantic name, original source type, or lifecycle.

Case 10 variant. Fact: "A verified indirect load reads a value from an effective address derived from a register or pointer." Hypothesis: "That value may represent a screen-state flag updated during sleep transitions." The stronger frequency and causality claims require separate runtime observation and writer provenance.

B.5. A final ten-question readiness exam

Complete this without looking up answers; each item tests a different failure mode.

1. Explain why MIPS16e mode can contain 4-byte instructions without changing the CPU's 32-bit register width.
2. For a short MIPS16e `BEQZ` at `0x1000` with imm8 `0x02`, calculate target and fall-through.
3. For a short MIPS16e `BNEZ` at `0x1000` with imm8 `0xFF`, calculate target.
4. Does the next instruction after a taken MIPS16e `BEQZ` run as a delay slot? Does the next legal instruction after a MIPS16e `JAL` run before the callee?
5. Which operand of `BEQZ v0,target` is being tested against zero?
6. Under O32, who preserves T0 if a caller needs it after a call? Who preserves S0 if a callee modifies it?
7. Why does `sw t0,0(sp)` write a register value into memory, and why must stack space be reserved first?
8. What does `0x4000FA00 + 0x204` equal, and what does the equality fail to prove?
9. Why can a Ghidra disassembly be wrong even when language is `MIPS:LE:32`?
10. Explain the difference between an observed byte sequence, a decoded instruction, and a semantic name such as `status->flags`.

Exam key: (1) Register width and instruction encoding length are independent. (2) Target `0x1006`, fall-through `0x1002`. (3) `0xFF` means -1 halfword → target `0x1000`. (4) No; yes. (5) V0. (6) Caller; callee. (7) Store direction and stack memory ownership. (8) `0x4000FC04`, not a structure or semantic identity. (9) ISA mode and instruction boundaries may be wrong. (10) Bytes are observation, decode is mode-dependent interpretation, semantic type is a claim requiring evidence.

SOURCES, SCOPE, AND VERIFICATION

[M1] Primary architecture manual. MIPS Technologies, *MIPS32 Architecture for Programmers, Volume IV-a: The MIPS16e Application-Specific Extension to the MIPS32 Architecture*, document MD00076, revision 2.63 (July 16, 2013). Especially **§3.6** register mapping, **§§3.7-3.8** ISA-mode switching, **§3.13** control-transfer delay rules, **§§3.14-3.17** formats and endianness, **BEQZ** pp. 68-69, **BNEZ** p. 70, **JAL** p. 83, **JALR** p. 84, **JALRC** p. 85, **JALX** pp. 86-87, **LW (PC-relative)** pp. 103-104, and **SAVE** pp. 123 onward. Available: https://docs.alexrp.com/mips/mips32_mips16e.pdf.

[G1] Ghidra's MIPS disassembly implementation. NSA Ghidra source code, `MipsDisassembleCommand.java`, describes `ISA_MODE` context and 2-byte MIPS16e versus 4-byte conventional MIPS alignment. https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Features/Base/src/main/java/ghidra/app/cmd/disassemble/MipsDisassembleCommand.java.

[G2] Ghidra processor specification. NSA Ghidra source, `mips.sinc`, defines context behavior including `ISA_MODE` and MIPS16e-related encoding information. https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Processors/MIPS/data/languages/mips.sinc.

[G3] Ghidra MIPS16 decoder implementation. NSA Ghidra source, `mips16.sinc`, including additional MIPS16e2 support. The presence of a decoder rule does **not** independently verify support in the target chip. https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Processors/MIPS/data/languages/mips16.sinc.

[G4] Ghidra user documentation. Ghidra, *Disassembly* help pages, including processor options and per-address register/context selection. https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Features/Base/src/main/help/help/topics/DisassemblerPlugin/Disassembly.htm.

[G5] Ghidra p-code language reference. Ghidra, *P-Code Operation Reference*, especially `LOAD` and `STORE`. https://ghidra.re/ghidra_docs/languages/html/pcodedescription.html.

[A1] ABI teaching reference. Dyncall, *MIPS O32 32-bit Calling Convention*. Used for typical register roles and caller/callee convention, **not as proof** of the firmware's precise ABI. https://dyncall.org/docs/manual/manualse11.html.

Scope and validation: Architectural rules and exact **short MIPS16e BEQZ/BNEZ/PC-relative LW** encodings in this companion were checked against MIPS16e manual format descriptions. The exercises deliberately distinguish **verified opcode constructions** from **schematic pseudocode**. Worked numerical branch and stack values can be reproduced arithmetically. Ghidra UI details can change by version. No claim is made that the target firmware has been loaded, dynamically executed, or verified at runtime by this document.

Next after the companion: Use one real MIPS16e function from your firmware, identify its bytes, prove the decode mode, trace a single caller and callee, and write a one-page evidence ledger. Do not advance to privileged exception machinery or debugger patching until these foundations are repeatable.